

Removing and adding network connections with Recursive Error Minimization (REM) equations

Wayne E. Simon and Jeffrey R. Carter

Martin Marietta Astronautics Group
MS DC4460
P. O. Box 179
Denver, CO 80201

ABSTRACT

One of the key features of Recursive Error Minimization (REM) equations is the efficient computation of the second derivative of mean square error with respect to each connection. The approximate integration of this derivative provides an estimate of the effect of removing or adding connections. A network with a minimum number of connections can then be found for a specific learning task. This has two important consequences. First, the explanation of network decisions is much simpler with a minimum net. Second, the computational load is a function of the number of connections. Results are presented for learning the English alphabet and, for a simpler task, learning the first seven letters of the alphabet.

1. INTRODUCTION

Determination of the minimum set of connections of a neural network is important for two distinct reasons. First, explaining the decisions of a neural network (determination of the explicit rules) is impractical with highly redundant nodes and connections. Explanation is vital because neural networks will not be used to make decisions with important consequences without being required to demonstrate the basis of the decisions (explicit rules). Second, for a serial machine, response time is proportional to the number of connections, and learning time is proportional to its square. For a parallel machine, hardware complexity is proportional to the number of connections, response time is constant, and learning time is proportional to the number of connections. This work demonstrates the ability of Recursive Error Minimization (REM) equations to solve part of this problem by removing unnecessary connections. The second part of the task, the fusion of correlated nodes, will require an extension of the REM equations.

One of the key features of REM equations, and a primary factor in the learning speed, is efficient computation of the second derivative of recursive mean square error with respect to each connection. An evaluation of the functional form of this derivative makes possible approximate integration, providing an estimate of the effect of adding or removing connections on the mean square error. It is then possible to remove all connections whose effect is less than a threshold value, but still provide for the re-establishment of connections, if necessary, as learning progresses.

In our previous papers, we derived REM learning equations for neural networks and applied them, first to the simple "exclusive-or" problem¹ and then to more complex and realistic problems, such as learning the English alphabet².

Equations 1, 2, and 3 describe the propagation of node state forward and the propagation of error and derivative backward. Equation 4 gives the rule for correcting the value of connections. The numerator of Equation 4 is, except for the method of presenting the constants, equivalent to the "generalized delta rule." The denominator is the second derivative of the mean square error with respect to the specific connection and has a lower bound of 0.000001 to prevent computational difficulties.

$$Y_d = \tanh\left(\frac{1}{2} \sum_c Y_c w_{cd}\right) \quad (1)$$

$$E_c^* = \left(\frac{1 - Y_c^2}{2}\right) \cdot \left[E_c + \sum_d w_{cd} \cdot E_d^*\right] \quad (2)$$

$$H_c^* = \left(\frac{1 - Y_c^2}{2}\right) \cdot \left[H_c + \sum_d w_{cd} \cdot H_d^*\right] \quad (3)$$

$$\Delta w_{cd} = \frac{RM_Q\left[\frac{\beta}{P} \cdot Y_c \cdot E_d^*\right]}{RM_P[(Y_c \cdot H_d^*)^2]} \quad (4)$$

Y_d = state of a node of the d^{th} layer

c, d = unspecified layers in a neural network

w_{cd} = connection from a node of layer c to a node of layer d

E_c^* = generalized error of a node of layer c (equivalent to the delta of the "generalized delta rule")

E_c = $D_c - Y_c$, if the node has a desired state (D_c)

= 0, otherwise

H_c^* = generalized derivative of a node of layer c

H_c = 1, if the node has a desired state (D_c)

= 0, otherwise

β = learning rate parameter (≈ 1.0)

$RM_P[\dots]$ = recursive mean of parameter in brackets with P as parametric memory

Finally, Equation 5 incorporates the technique which guarantees convergence to a global minimum. The initial desired output is defined to be the initial output of the network, and is then smoothly changed to the final desired output as learning progresses. If the rate of change of the desired output is sufficiently

small, the network is always in the neighborhood of a global minimum, and will still be in the neighborhood of a global minimum when the desired output has transitioned to the final desired output.

$$D_{q,l}^n = D_{q,l}^{n-1} \cdot \left(\frac{R-1}{R} \right) + D_{q,l}^{\infty} \cdot \left(\frac{1}{R} \right) \quad (5)$$

$D_{q,l}^0$ = initial desired output (= initial observed output) of the l^{th} output node to the q^{th} experience

$D_{q,l}^{\infty}$ = final desired output of the l^{th} output node to the q^{th} experience

R = parametric memory of initial desired output

2. ANALYSIS

Note that

$$RM_p \left[(Y_c \cdot H_d^*)^2 \right]$$

is the second derivative of the mean square error of the network response with respect to the connection from node "c" to node "d." In order to select connections for removal from or addition to the network, we need an estimate of the change in mean square error for removal or addition of a specific connection. This means two integrations with respect to the value of the connection in a very non-linear, densely connected system. Further, we already know that the second derivative goes exponentially to zero as the mean square error goes to zero. Thus the more accurately the network responds, the more nearly the current values of the second derivatives approach zero. This indicates a path to the estimate. Inside the recursive mean operation, we can write Equation 6.

$$\frac{\partial^2 E_d^2}{\partial w_{cd}^2} = (Y_c H_d^*)^2 \quad (6)$$

Taking a derivative with respect to the connection gives Equation 7.

$$\frac{\partial^3 E_d^2}{\partial w_{cd}^3} = -2Y_c Y_d (Y_c H_d^*)^2 \quad (7)$$

With this information, the second derivative can now be approximated as Equation 8.

$$\frac{\partial^2 E_d^2}{\partial w_{cd}^2} \approx (Y_c H_d^*)_{w_{cd}=0}^2 \cdot e^{-2Y_c Y_d w_{cd}} \quad (8)$$

This approximation can then be integrated twice, with the appropriate limits of the first derivative, to obtain Equation 9.

$$\Delta E_{cd}^2 \approx \frac{(Y_c H_d^*)^2}{(2Y_c Y_d)^2} \cdot (1 - (1 + \psi)e^{-\psi}) \quad (9)$$

where:

ΔE_{cd} = total effect of connection from c to d on system square error

$$\psi = 2Y_c Y_d w_{cd} \Delta w_{cd}$$

$$w_{cd} \Delta w_{cd} = w_{cd} + \Delta w_{cd}$$

Rewriting in terms of the current value of $(Y_c H_d^*)^2$ gives Equation 10.

$$\Delta E_{cd}^2 \approx (Y_c H_d^*)^2 \cdot e^{2Y_c Y_d w_{cd}} \cdot \left[\frac{(1 - (1 + \psi)e^{-\psi})}{(2Y_c Y_d)^2} \right] \quad (10)$$

Note that if Y_c or Y_d approach zero, both the numerator and the denominator of the bracketed ratio of Equation 10 approach zero. Evaluating with l'Hospital's rule gives Equation 11.

$$\Delta E_{cd}^2 \approx (Y_c H_d^*)^2 \cdot e^{2Y_c Y_d w_{cd}} \cdot \frac{(w_{cd} + \Delta w_{cd})^2}{2} \quad (11)$$

If the denominator of Equation 10 is less than a small constant, say 0.000001, use Equation 11 to estimate the effect of the connection on mean square error. Otherwise, use Equation 10.

Finally, going back to a recursive mean, but providing for the possibility of a different parametric memory, gives Equation 12.

$$G_{cd} = RM_S[\Delta E_{cd}^2] \quad (12)$$

A parameter with the dimension of square error, EC , is specified for the purpose of removing and adding connection to the network. The initial value of G_{cd} is set to $2EC$. As long as G_{cd} stays above EC , the value of the connection is updated with the current correction. If G_{cd} crosses from above to below EC , both G_{cd} and the connection, w_{cd} , are set to zero. If G_{cd} stays below EC , no updates are performed. If G_{cd} crosses EC from below to above, G_{cd} is set to $2EC$ and w_{cd} is set to Δw_{cd} . The effect of these rules is to remove all those connections whose effect is less than EC , with a hysteresis in the process, as a function of S , which prevents rapid switching of connections in and out of the network.

If the connection removal process sets all the connections to a node to zero, its state goes to zero. If all the connections from a node are set to zero, its generalized derivative goes to zero. In either case, the second derivative of the connections can go to zero. To remedy this problem, we take a hint from biological networks and make the net input to a node uncertain. That is, a uniformly distributed random variable with zero mean and a specified range is added to the net input. The result is to maintain a proper calculation of the second derivative, even though the net input is effectively zero.

Note that the connection removal process may effectively remove nodes by setting to zero either all the connections to a node, or all the connections from a node, but it does not result in a minimum number of nodes. The consolidation of nodes with highly correlated node states requires a different technique.

3. RESULTS

The network chosen for a demonstration of this technique is the 20-20-5 network used to learn the English alphabet. The feed-forward networks used in this work connect the input to both the intermediate nodes and the output nodes. Figure 1 shows the reduction in connections which is obtained. The strength of the identifications is, of course, reduced. This network started with 625 connections and ended with 148, after 300 epochs of training. A 20-10-5 network ended with 155, and a 20-5-5 network ended with 159 connections. This gives the interesting speculation that the information needed to recognize this alphabet can be contained in about 150 connections.

Finally, a simpler learning problem is posed to show that the minimum number of connections is a function of the complexity of the learning task. A 20-20-3 network (543 connections) is used to learn the first seven letters of the alphabet. Figure 2 shows the removal of connections. The first seven letters can be identified with only 12 connections, about 2% of the original 20-20-3 network.

Examination of this run shows that, of the twenty input nodes, only seven are still connected. Four have only one connection each to an output node. One has connections to two output nodes. The remaining two each have a single connection to an intermediate node. These two intermediate nodes are each connected to two output nodes. The final configuration produced by removing connections is, of course, not necessarily unique. Another run ended with 14 connections, but only one hidden node.

This suggests that this learning problem can be handled without intermediate nodes. A perceptron model using only input and output nodes can be processed to remove connections, resulting in eight to eleven connections, depending on initial conditions.

4. CONCLUSIONS

A technique has been developed and demonstrated to reduce the number of connections in a network by a factor of four to a factor of fifty. The possible reduction is a function of the initial complexity of the network and the complexity of the learning task. Further, the development of a technique to combine correlated nodes may reduce the number of connections even more. Reduction

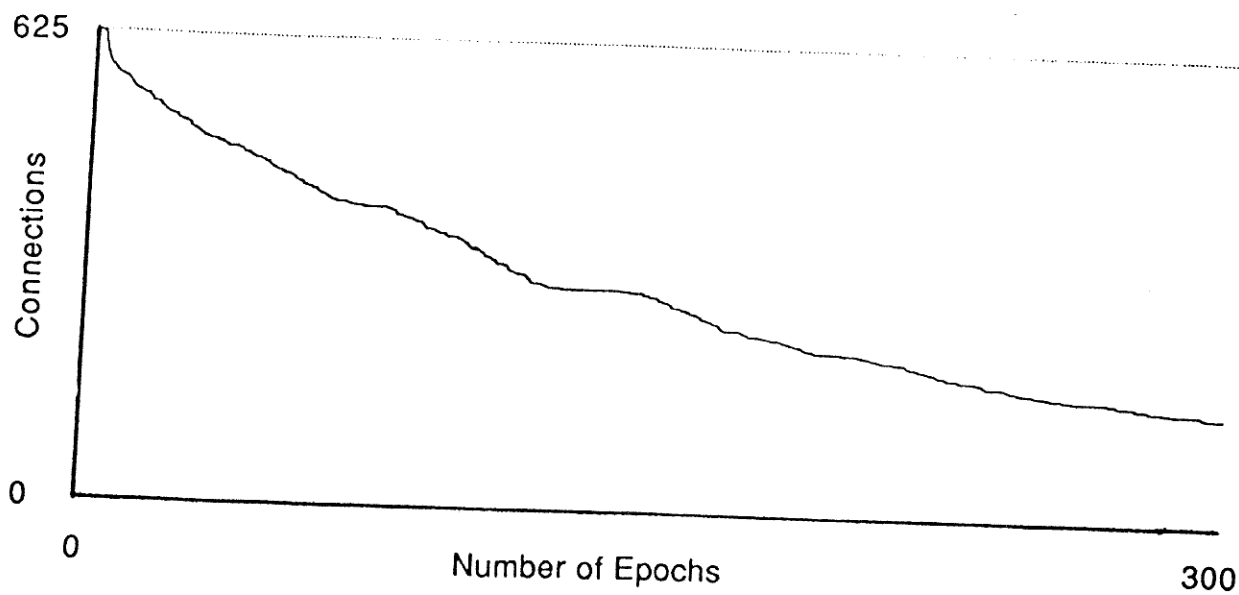


Figure 1 Reducing Connections in a 20-20-5 Network Learning the Alphabet

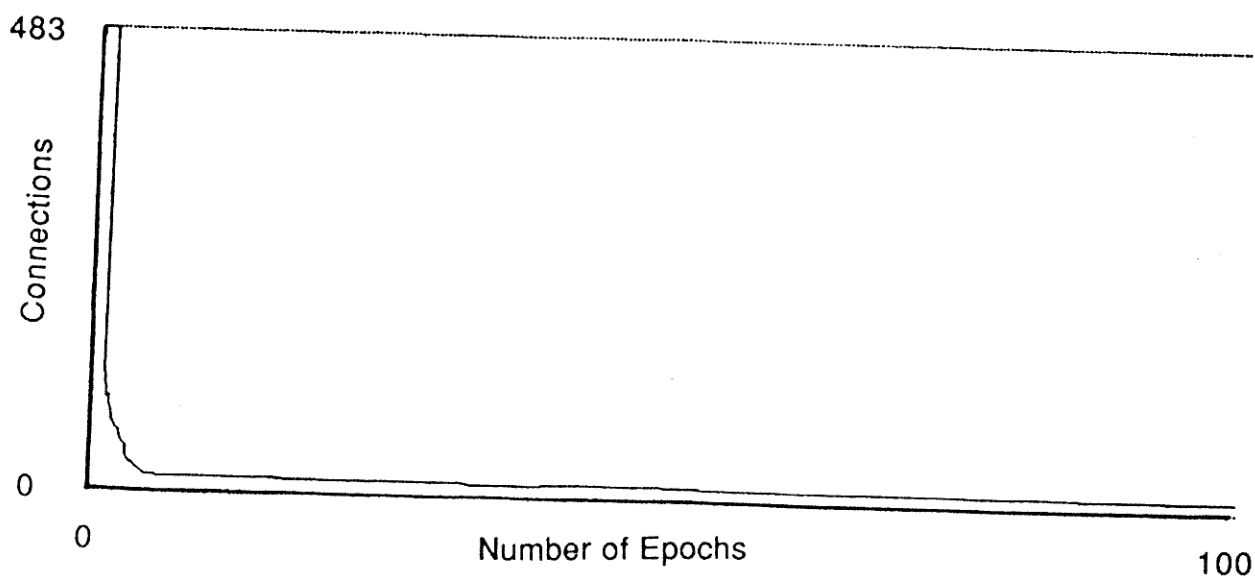


Figure 2 Reducing Connections in a 20-20-3 Network Learning the First Seven Letters of the Alphabet

of the average number of connections per node to the range of one to three makes the determination of the explicit rules describing the network decisions quite a simple task.

5. REFERENCES

1. Simon, W., and J. Carter, "Back Propagation Learning Equations from the Minimization of Recursive Error," *Proceedings of the IEEE International Conference on Systems Engineering*, IEEE, 1989.

2. Simon, W., and J. Carter, "Learning to Identify Letters with REM Equations," *Proceedings of the International Joint Conference on Neural Networks*, Vol. I, Lawrence Erlbaum Associates, 1990.